



14.8. Cocktail Sort & Odd-Even Sort

Previous Section:

 [14.7. Shell Sort](#)

Key Section Review:

 [14.1. Bubble Sort & Comb Sort](#)

Contents:

[1. Cocktail Sort](#)

[2. Odd-Even Sort](#)

[3. Implementation](#)

[Summary](#)

[References](#)

Cocktail Sort and Odd-Even Sort are both **variations of Bubble Sort**. Like Bubble Sort, they repeatedly compare adjacent elements and swap them when they are in the wrong order. Their goal is to improve Bubble Sort by modifying the way comparisons are performed.

- **Cocktail Sort** improves Bubble Sort by traversing the array in **both directions**, allowing the largest elements to move toward the end while the smallest elements move toward the beginning during the same iteration.
- **Odd-Even Sort** divides each pass into two phases: one compares adjacent pairs starting at odd indices, while the other compares adjacent pairs starting at even indices. These two phases are repeated until the array is completely sorted.

Although both algorithms still have the same worst-case time complexity as Bubble Sort, they usually perform fewer unnecessary passes on partially sorted data.

1. Cocktail Sort

Cocktail Sort (also called **Bidirectional Bubble Sort** or **Shaker Sort**) improves Bubble Sort by scanning the array in **both directions** during each iteration.

A complete iteration consists of:

- A **left-to-right pass**, which moves the largest unsorted element to the end.
- A **right-to-left pass**, which moves the smallest unsorted element to the beginning.

As the algorithm continues:

- The largest element reaches its final position.
- The smallest element reaches its final position.
- Then the second-largest and second-smallest elements are positioned.
- This process repeats until the array becomes sorted.

Compared to Bubble Sort, Cocktail Sort often requires fewer passes because both ends of the array are sorted simultaneously.

Time Complexity: $O(n^2)$; **Space Complexity:** $O(1)$

- **Demonstration of Cocktail Sort:** Consider the following array.

8	3	2	7	14	6	12	5
---	---	---	---	----	---	----	---

- **Pass 1 (Left → Right)**

Compare adjacent elements from left to right.

8 > 3 -> Swap

--	--	--	--	--	--	--	--

3	8	2	7	14	6	12	5
---	---	---	---	----	---	----	---

8 > 2 -> Swap

3	2	8	7	14	6	12	5
---	---	---	---	----	---	----	---

8 > 7 -> Swap

3	2	7	8	14	6	12	5
---	---	---	---	----	---	----	---

8 < 14 -> No Swap

3	2	7	8	14	6	12	5
---	---	---	---	----	---	----	---

14 > 6 -> Swap

3	2	7	8	6	14	12	5
---	---	---	---	---	----	----	---

14 > 12 -> Swap

3	2	7	8	6	12	14	5
---	---	---	---	---	----	----	---

14 > 5 -> Swap

3	2	7	8	6	12	5	14
---	---	---	---	---	----	---	----

Array becomes

3	2	7	8	6	12	5	14
---	---	---	---	---	----	---	----

The largest element (**14**) is now in its correct position.

- Pass 1 (Right → Left)

Traverse the array in reverse.

5 < 14 -> No Swap

3	2	7	8	6	12	5	14
---	---	---	---	---	----	---	----

12 > 5 -> Swap

3	2	7	8	6	5	12	14
---	---	---	---	---	---	----	----

6 > 5 -> Swap

3	2	7	8	5	6	12	14
---	---	---	---	---	---	----	----

8 > 5 -> Swap

3	2	7	5	8	6	12	14
---	---	---	---	---	---	----	----

7 > 5 -> Swap

3	2	5	7	8	6	12	14
---	---	---	---	---	---	----	----

2 < 5 -> No Swap

3	2	5	7	8	6	12	14
---	---	---	---	---	---	----	----

3 > 2 -> Swap

2	3	5	7	8	6	12	14
---	---	---	---	---	---	----	----

Array becomes

2	3	5	7	8	6	12	14
---	---	---	---	---	---	----	----

The smallest element (**2**) is now in its correct position.

- Pass 2 (Left → Right)

Continue sorting only the unsorted middle section.

8 > 6 -> Swap

2	3	5	7	6	8	12	14
---	---	---	---	---	---	----	----

- Pass 2 (Right → Left)

7 > 6 Swap

2	3	5	6	7	8	12	14
---	---	---	---	---	---	----	----

Array becomes

2	3	5	6	7	8	12	14
---	---	---	---	---	---	----	----

The array is now completely sorted.

2. Odd-Even Sort

Odd-Even Sort is another variation of Bubble Sort that divides each iteration into **two phases**.

- **Odd Phase:** Compare adjacent elements beginning at odd indices.
- **Even Phase:** Compare adjacent elements beginning at even indices.

The two phases are repeated until no swaps occur.

Unlike Bubble Sort, which compares every adjacent pair during one pass, Odd-Even Sort separates the comparisons into alternating groups.

Time Complexity: $O(n^2)$; **Space Complexity:** $O(1)$

- Demonstration of Odd-Even Sort: Consider the same array.

8	3	2	7	14	6	12	5
---	---	---	---	----	---	----	---

Indices 0 1 2 3 4 5 6 7

- Phase 1 — Odd Indices: Compare (1, 2); (3, 4); (5, 6)

3 > 2 -> Swap

8	2	3	7	14	6	12	5
---	---	---	---	----	---	----	---

7 < 14 -> No Swap

8	3	2	7	14	6	12	5
---	---	---	---	----	---	----	---

6 < 12 -> No Swap

8	3	2	7	14	6	12	5
---	---	---	---	----	---	----	---

Array becomes

8	2	3	7	14	6	12	5
---	---	---	---	----	---	----	---

- Phase 2 — Even Indices: Compare (0, 1); (2, 3); (4, 5); (6, 7)

8 > 2 -> Swap

2	8	3	7	14	6	12	5
---	---	---	---	----	---	----	---

3 < 7 -> No Swap

2	8	3	7	14	6	12	5
---	---	---	---	----	---	----	---

14 > 6 -> Swap

2	8	3	7	6	14	12	5
---	---	---	---	---	----	----	---

12 > 5 -> Swap

2	8	3	7	6	14	5	12
---	---	---	---	---	----	---	----

Array becomes:

2	8	3	7	6	14	5	12
---	---	---	---	---	----	---	----

- **Continue the Process**

The algorithm alternates between **Odd Phase → Even Phase → Odd Phase → Even Phase**, until an entire iteration completes without any swaps.

Swaps occurring in the odd-indices phase:

8 > 3 -> Swap

2	3	8	7	6	14	5	12
---	---	---	---	---	----	---	----

7 > 6 -> Swap

2	3	8	6	7	14	5	12
---	---	---	---	---	----	---	----

14 > 5 -> Swap

2	3	8	6	7	5	14	12
---	---	---	---	---	---	----	----

Swaps occurring in the even-indices phase:

8 > 6 -> Swap

2	3	6	8	7	5	14	12
---	---	---	---	---	---	----	----

7 > 5 -> Swap

2	3	6	8	5	7	14	12
---	---	---	---	---	---	----	----

14 > 12 -> Swap

2	3	6	8	5	7	12	14
---	---	---	---	---	---	----	----

Swaps occurring in the odd-indices phase:

8 > 5 -> Swap

--	--	--	--	--	--	--	--

2	3	6	5	8	7	12	14
---	---	---	---	---	---	----	----

Swaps occurring in the even-indices phase:

6 > 5 -> Swap

2	3	5	6	8	7	12	14
---	---	---	---	---	---	----	----

8 > 7 -> Swap

2	3	5	6	7	8	12	14
---	---	---	---	---	---	----	----

Eventually, the array becomes:

2	3	5	6	7	8	12	14
---	---	---	---	---	---	----	----

3. Implementation

Both Cocktail Sort and Odd-Even Sort are simple modifications of Bubble Sort.

Cocktail Sort repeatedly performs a forward pass followed by a backward pass, gradually shrinking the unsorted portion from both ends until the array is sorted.

- Implementation of Cocktail Sort

```
def cocktail_sort(a):
    n = len(a)
    for i in range(n - 1, 0, -1):
        is_swapped = False
        for j in range(i, 0, -1): # Visit the sequence from
            # right to left
            if a[j] < a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
                is_swapped = True
        for j in range(i): # Visit the sequence from left
            # to right
```



```

        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            is_swapped = True
    if not is_swapped: # If there is no swap in the entire iteration
        return a # The array is sorted

```

Odd-Even Sort repeatedly alternates between comparing adjacent pairs that begin at odd indices and those that begin at even indices. The algorithm terminates once an entire odd-even cycle completes without performing any swaps.

- Implementation of Odd-Even Sort

```

def odd_even_sort(a): # Time Complexity: O(N^2)
    n = len(a)
    for i in range(n): # Iterate through array
        is_sorted = True
        for j in range(1, n - 1, 2): # Compare and swap the odd indexed elements
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                is_sorted = False # The array is not yet sorted
        for j in range(0, n - 1, 2): # Compare and swap the even indexed elements
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                is_sorted = False # The array is not yet sorted
        if is_sorted: # If the array is already sorted
            break
    return a

```

Both algorithms are implemented **in-place**, requiring only constant extra memory.

Summary

Cocktail Sort and Odd-Even Sort are two improvements of Bubble Sort that attempt to reduce unnecessary work while maintaining the simplicity of adjacent-element comparisons.

- Cocktail Sort traverses the array in both directions during each iteration, allowing the smallest and largest elements to move into position simultaneously.
- Odd-Even Sort divides each iteration into alternating odd and even comparison phases, making it particularly suitable for parallel implementations.

Although both algorithms still have a worst-case time complexity of $O(n^2)$ and a space complexity of $O(1)$, they generally perform better than the standard Bubble Sort on partially sorted data while remaining easy to understand and implement.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/cocktail-sort/>

<https://www.geeksforgeeks.org/dsa/odd-even-sort-brick-sort/>

Next Section:

 [15. Hash Tables](#)